DIGITAL ELECTRONICS ENGINEERING (DEE) LAB MANUAL

Functions, Boolean Equations & Truth Tables for All 50 Logic Circuits

Complete reference guide corresponding to the 50 indexed DEE Lab topics (S.No. 1 to 50) and downloadable Logisim (`.circ`) schematics.

Master Index of All 50 Logic Circuits

S.No.	Topic Name	Category	Lab Page
1	1×4 Demultiplexer	Mux & Demux	1
2	2×4 Decoder	Encoders & Decoders	1
3	3×8 Decoder	Encoders & Decoders	2
4	4×1 Multiplexer	Mux & Demux	3
5	8×1 Multiplexer	Mux & Demux	3
6	Octal to Binary Encoder	Encoders & Decoders	5
7	Decimal to BCD Encoder	Encoders & Decoders	6
8	2×1 Multiplexer	Mux & Demux	7
9	3-Bit Even Parity Generator using NAND Gate	Parity & Converters	7
10	4-Bit Even Parity Generator using NAND Gate	Parity & Converters	8
11	AND Gate using NAND Gate	Universal Gate Conversions	8
12	AND Gate using NOR Gate	Universal Gate Conversions	8
13	D Flip-Flop using NAND Gate	Latches & Flip-Flops	9
14	Excess-3 Code Conversion from BCD	Parity & Converters	9
15	Full Adder	Adders & Subtractors	10
16	Full Subtractor	Adders & Subtractors	10
17	Half Adder	Adders & Subtractors	11
18	Half Subtractor	Adders & Subtractors	11
19	Implementation of Three-Variable Boolean Expression using Basic Gates	Boolean Expressions	11
20	Implementation of Three-Variable Boolean Expression using NAND Gates	Boolean Expressions	12

S.No.	Topic Name	Category	Lab Page
21	JK Flip-Flop using NAND Gate	Latches & Flip-Flops	12
22	Hexadecimal to Binary Encoder	Encoders & Decoders	13
23	NAND Gate using NOR Gate	Universal Gate Conversions	14
24	NOR Gate using NAND Gate	Universal Gate Conversions	14
25	OR Gate using NAND Gate	Universal Gate Conversions	15
26	OR Gate using NOR Gate	Universal Gate Conversions	15
27	SR Flip-Flop using NAND Gate	Latches & Flip-Flops	15
28	SR Flip-Flop using NOR Gate	Latches & Flip-Flops	16
29	SR Latch using NAND Gate	Latches & Flip-Flops	16
30	SR Latch using NOR Gate	Latches & Flip-Flops	16
31	Seven Segment Display (3-Bit)	Encoders & Decoders	17
32	XNOR Gate using NAND Gate	Universal Gate Conversions	18
33	XNOR Gate using NOR Gate	Universal Gate Conversions	18
34	XOR Gate using NAND Gate	Universal Gate Conversions	18
35	XOR Gate using NOR Gate	Universal Gate Conversions	18
36	AND Gate	Basic Logic Gates	19
37	OR Gate	Basic Logic Gates	19
38	NOT Gate	Basic Logic Gates	19
39	XOR Gate	Basic Logic Gates	19
40	XNOR Gate	Basic Logic Gates	19
41	NOR Gate	Basic Logic Gates	20
42	NAND Gate	Basic Logic Gates	20
43	T Flip-Flop	Latches & Flip-Flops	20
44	Implementation of Three-Variable Boolean Expression using NOR Gates	Boolean Expressions	20

S.No.	Topic Name	Category	Lab Page
45	Implementation of Four-Variable Boolean Expression using Basic Gates	Boolean Expressions	21
46	Implementation of Four-Variable Boolean Expression using NAND Gate	Boolean Expressions	22
47	NOT Gate using NAND Gate	Universal Gate Conversions	22
48	NOT Gate using NOR Gate	Universal Gate Conversions	22
49	Implementation of Two-Variable Boolean Expression using Basic Gates	Boolean Expressions	23
50	Implementation of Two-Variable Boolean Expression using NAND Gate	Boolean Expressions	23

1. 1×4 Demultiplexer (Mux & Demux · Lab Page 1)

Function: Routes a single data input line (D) to one of four output lines (Y0, Y1, Y2, Y3) selected by the 2-bit binary select inputs (S1, S0). Constructed using 2 NOT gates and 4 three-input AND gates.

> **Boolean Expression:** $Y_0 = \overline{S_1} \cdot \overline{S_0} \cdot D$, $Y_1 = \overline{S_1} \cdot S_0 \cdot D$, $Y_2 = S_1 \cdot \overline{S_0} \cdot D$, $Y_3 = S_1 \cdot S_0 \cdot D$

D	S1	S0	Y0	Y1	Y2	Y3	Selected Output
0	X	X	0	0	0	0	All outputs 0 (Data = 0)
1	0	0	1	0	0	0	Y0 = D = 1
1	0	1	0	1	0	0	Y1 = D = 1
1	1	0	0	0	1	0	Y2 = D = 1
1	1	1	0	0	0	1	Y3 = D = 1

2. 2×4 Decoder (Encoders & Decoders · Lab Page 1)

Function: Decodes a 2-bit binary input code (A1, A0) into four mutually exclusive active-high minterm outputs (Y0 to Y3). Constructed with 2 NOT gates and 4 two-input AND gates.

Boolean Expression: $Y_0 = \overline{A_1} \cdot \overline{A_0}, \quad Y_1 = \overline{A_1} \cdot A_0, \quad Y_2 = A_1 \cdot \overline{A_0}, \quad Y_3 = A_1 \cdot A_0$

A1	A0	Y0	Y1	Y2	Y3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

3. 3×8 Decoder (Encoders & Decoders · Lab Page 2)

Function: Decodes a 3-bit binary address (A2, A1, A0) into eight one-hot active-high minterm outputs (Y0 to Y7, octal decoding). Constructed using 3 NOT gates and 8 three-input AND gates.

Boolean Expression: $Y_k = m_k \quad (\text{e.g., } Y_0 = \overline{A_2} \cdot \overline{A_1} \cdot \overline{A_0}, \ldots, Y_7 = A_2 \cdot A_1 \cdot A_0)$

A2	A1	A0	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

4. 4×1 Multiplexer (Mux & Demux · Lab Page 3)

Function: Selects one of four data inputs (I0, I1, I2, I3) and routes it to the single output (Y) based on the 2-bit select lines (S1, S0). Constructed using 2 NOT gates, 4 three-input AND gates, and an OR gate tree.

Boolean Expression: $Y = \overline{S_1}\cdot\overline{S_0}\cdot I_0 + \overline{S_1}\cdot S_0\cdot I_1 + S_1\cdot\overline{S_0}\cdot I_2 + S_1\cdot S_0\cdot I_3$

S1	S0	Selected Input	Output Y
0	0	Channel 0	Y = I0
0	1	Channel 1	Y = I1
1	0	Channel 2	Y = I2
1	1	Channel 3	Y = I3

5. 8×1 Multiplexer (Mux & Demux · Lab Page 3)

Function: Selects one of eight data inputs (I0 through I7) and routes it to a single output (Y) according to the 3-bit select address (S2, S1, S0). Built using 3 NOT gates, 8 product-term AND gates, and an OR combiner.

Boolean Expression: $Y = \sum_{k=0..7}(m_k \cdot I_k) = \overline{S_2}\overline{S_1}\overline{S_0}I_0 + \overline{S_2}\overline{S_1}S_0I_1 + \ldots + S_2S_1S_0I_7$

S2	S1	S0	Minterm Selected	Output Y
0	0	0	m0	Y = I0
0	0	1	m1	Y = I1
0	1	0	m2	Y = I2
0	1	1	m3	Y = I3
1	0	0	m4	Y = I4
1	0	1	m5	Y = I5
1	1	0	m6	Y = I6
1	1	1	m7	Y = I7

6. Octal to Binary Encoder (Encoders & Decoders · Lab Page 5)

Function: Converts one-hot octal inputs (D0 to D7) into a 3-bit binary code (Y2, Y1, Y0). Each output bit is formed by an OR gate summing the octal lines whose binary index has a 1 in that bit position.

> **Boolean Expression:** $Y_2 = D_4 + D_5 + D_6 + D_7$, $Y_1 = D_2 + D_3 + D_6 + D_7$, $Y_0 = D_1 + D_3 + D_5 + D_7$

Active Input	D7	D6	D5	D4	D3	D2	D1	D0	Y2	Y1	Y0
D0 = 1	0	0	0	0	0	0	0	1	0	0	0
D1 = 1	0	0	0	0	0	0	1	0	0	0	1
D2 = 1	0	0	0	0	0	1	0	0	0	1	0
D3 = 1	0	0	0	0	1	0	0	0	0	1	1
D4 = 1	0	0	0	1	0	0	0	0	1	0	0
D5 = 1	0	0	1	0	0	0	0	0	1	0	1
D6 = 1	0	1	0	0	0	0	0	0	1	1	0
D7 = 1	1	0	0	0	0	0	0	0	1	1	1

7. Decimal to BCD Encoder (Encoders & Decoders · Lab Page 6)

Function: Encodes ten active-high decimal digit lines (D0 through D9) into a 4-bit Binary-Coded Decimal (BCD) output (Y3, Y2, Y1, Y0) using OR gates.

Boolean Expression: $Y_3 = D_8 + D_9,\quad Y_2 = D_4 + D_5 + D_6 + D_7,\quad Y_1 = D_2 + D_3 + D_6 + D_7,\quad Y_0 = D_1 + D_3 + D_5 + D_7 + D_9$

Decimal Digit	Active Input Line	Y3 (MSB)	Y2	Y1	Y0 (LSB)
0	D0 = 1 (others 0)	0	0	0	0
1	D1 = 1 (others 0)	0	0	0	1
2	D2 = 1 (others 0)	0	0	1	0
3	D3 = 1 (others 0)	0	0	1	1
4	D4 = 1 (others 0)	0	1	0	0
5	D5 = 1 (others 0)	0	1	0	1
6	D6 = 1 (others 0)	0	1	1	0
7	D7 = 1 (others 0)	0	1	1	1
8	D8 = 1 (others 0)	1	0	0	0
9	D9 = 1 (others 0)	1	0	0	1

8. 2×1 Multiplexer (Mux & Demux · Lab Page 7)

Function: Selects between two input data lines (I0 and I1) controlled by a single select input (S0). When S0 = 0, Y = I0; when S0 = 1, Y = I1. Built with 1 NOT gate, 2 AND gates, and 1 OR gate.

Boolean Expression: $Y = (\overline{S_0} \cdot I_0) + (S_0 \cdot I_1)$

S0	I0	I1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

9. 3-Bit Even Parity Generator using NAND Gate (Parity & Converters · Lab Page 7)

Function: Generates an even parity bit (P) for a 3-bit message (A, B, C) such that the total number of 1s in (A, B, C, P) is always even. Implemented using two cascaded 4-NAND XOR modules (8 NAND gates total).

Boolean Expression: $P = A \oplus B \oplus C$

A	B	C	P (Even Parity)
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

10. 4-Bit Even Parity Generator using NAND Gate (Parity & Converters · Lab Page 8)

Function: Computes the even parity bit (P) for a 4-bit input word (A, B, C, D) so the 5-bit transmitted code contains an even number of 1s. Built strictly from three 4-NAND XOR stages (12 NAND gates).

Boolean Expression: $P = (A \oplus B) \oplus (C \oplus D)$

A	B	C	D	P (Even Parity)
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

11. AND Gate using NAND Gate (Universal Gate Conversions · Lab Page 8)

Function: Realizes the logical AND operation using universal NAND gates by cascading a 2-input NAND gate with a single-input NAND inverter (inputs tied together).

Boolean Expression: $Y = \overline{A \cdot B} = (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B) = A \cdot B$

A	B	n1 = ~(A.B)	Y = A . B
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

12. AND Gate using NOR Gate (Universal Gate Conversions · Lab Page 8)

Function: Realizes the logical AND operation using universal NOR gates via De Morgan's law: two NOR inverters produce ~A and ~B, which feed a third NOR gate.

> **Boolean Expression: $Y = \overline{A + B} = (A \text{ NOR } A) \text{ NOR } (B \text{ NOR } B) = A \cdot B$**

A	B	~A	~B	Y = A . B
0	0	1	1	0
0	1	1	0	0
1	0	0	1	0
1	1	0	0	1

13. D Flip-Flop using NAND Gate (Latches & Flip-Flops · Lab Page 9)

Function: Eliminates the invalid S=R=1 state of the SR flip-flop by feeding D to the Set steering gate and ~D (via a NAND inverter) to the Reset steering gate. When CLK = 1, Q tracks D; when CLK = 0, Q holds its state.

> **Boolean Expression: $Q(t+1) = D \quad \text{(when CLK = 1)}; \quad Q(t+1) = Q(t) \quad \text{(when CLK = 0)}$**

CLK	D	Q(t+1)	Q_bar(t+1)	Operating Mode
0	X	Q(t)	Q_bar(t)	Hold (Memory / No Change)
1	0	0	1	Reset (Transparent Data 0)
1	1	1	0	Set (Transparent Data 1)

14. Excess-3 Code Conversion from BCD (Parity & Converters · Lab Page 9)

Function: Converts a 4-bit Binary-Coded Decimal (BCD) digit (B3, B2, B1, B0 for 0..9) into its self-complementing Excess-3 code (E3, E2, E1, E0) by adding binary 0011 (+3).

Boolean Expression: $E_3 = B_3 + B_2B_0 + B_2B_1$, $E_2 = \overline{B_2}B_1 + \overline{B_2}B_0 + B_2\overline{B_1}\,\overline{B_0}$, $E_1 = B_1B_0 + \overline{B_1}\,\overline{B_0}$, $E_0 = \overline{B_0}$

Decimal	B3	B2	B1	B0	E3	E2	E1	E0
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

15. Full Adder (Adders & Subtractors · Lab Page 10)

Function: Adds three 1-bit binary numbers (augend A, addend B, and input carry Cin) to produce a Sum bit and an output carry (Cout). Built from two Half Adders (2 XOR, 2 AND gates) and 1 OR gate.

Boolean Expression: $\mathbf{Sum} = A \oplus B \oplus C_{in}$, $C_{out} = (A \cdot B) + C_{in} \cdot (A \oplus B)$

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

16. Full Subtractor (Adders & Subtractors · Lab Page 10)

Function: Subtracts subtrahend bit (B) and previous borrow (Bin) from minuend bit (A), generating Difference (Diff) and output Borrow (Bout). Constructed with 2 XOR gates, 2 NOT gates, 2 AND gates, and 1 OR gate.

> **Boolean Expression:** $\mathbf{Diff = A \oplus B \oplus B_{in}}$, $\mathbf{B_{out} = (\overline{A} \cdot B) + (\overline{A \oplus B} \cdot B_{in})}$

A	B	Bin	Diff	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

17. Half Adder (Adders & Subtractors · Lab Page 11)

Function: Adds two 1-bit binary inputs (A and B) and produces a Sum bit and a Carry-out bit using 1 XOR gate and 1 AND gate.

> **Boolean Expression:** $\mathbf{Sum = A \oplus B}$, $\mathbf{Carry = A \cdot B}$

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

18. Half Subtractor (Adders & Subtractors · Lab Page 11)

Function: Subtracts bit B from bit A (A - B) to produce a Difference bit (Diff) and a Borrow bit using 1 XOR gate, 1 NOT gate, and 1 AND gate.

> **Boolean Expression:** $\mathbf{Diff = A \oplus B, \quad Borrow = \overline{A} \cdot B}$

A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

19. Implementation of Three-Variable Boolean Expression using Basic Gates

(Boolean Expressions · Lab Page 11)

Function: Implements a 3-variable Sum-of-Products (SOP) Boolean expression directly using basic inverters (NOT), product gates (AND), and a summing gate (OR).

> **Boolean Expression:** $\mathbf{Y(A, B, C) = (A \cdot B) + (\overline{A} \cdot C) + (B \cdot \overline{C})}$

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

20. Implementation of Three-Variable Boolean Expression using NAND Gates

(Boolean Expressions · Lab Page 12)

Function: Realizes the 3-variable SOP Boolean expression exclusively with universal NAND gates by replacing the AND-OR two-level network with a NAND-NAND network and NAND inverters.

Boolean Expression: $Y = \overline{A \cdot B \cdot A \cdot C \cdot B \cdot C} = (A \cdot B) + (\overline{A} \cdot C) + (B \cdot \overline{C})$

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

21. JK Flip-Flop using NAND Gate (Latches & Flip-Flops · Lab Page 12)

Function: Universal clocked flip-flop built with two 3-input steering NAND gates (incorporating cross-feedback from Q and Q_bar) driving an active-low NAND latch. Resolves the S=R=1 ambiguity by toggling the output when J=K=1.

Boolean Expression: $Q(t+1) = J \cdot \overline{Q(t)} + \overline{K} \cdot Q(t) \quad \text{(when CLK = 1)}$

CLK	J	K	Q(t+1)	Q_bar(t+1)	Operating Mode
0	X	X	Q(t)	Q_bar(t)	Hold (Clock Low)
1	0	0	Q(t)	Q_bar(t)	Hold (No Change)
1	0	1	0	1	Reset (Q = 0)
1	1	0	1	0	Set (Q = 1)
1	1	1	Q_bar(t)	Q(t)	Toggle (Complement)

22. Hexadecimal to Binary Encoder (Encoders & Decoders · Lab Page 13)

Function: Compresses 16 active-high hexadecimal input lines (H0 to H15, corresponding to 0×0..0xF) into a 4-bit binary output (Y3, Y2, Y1, Y0) using multi-input OR trees.

Boolean Expression: $Y_3 = \sum H_{8..15}$, $Y_2 = \sum H_{4..7,12..15}$, $Y_1 = \sum H_{2,3,6,7,10,11,14,15}$, $Y_0 = \sum H_{odd}$

Hex Digit	Active Input	Y3	Y2	Y1	Y0
0 (0×0)	H0 = 1	0	0	0	0
1 (0×1)	H1 = 1	0	0	0	1
2 (0×2)	H2 = 1	0	0	1	0
3 (0×3)	H3 = 1	0	0	1	1
4 (0×4)	H4 = 1	0	1	0	0
5 (0×5)	H5 = 1	0	1	0	1
6 (0×6)	H6 = 1	0	1	1	0
7 (0×7)	H7 = 1	0	1	1	1
8 (0×8)	H8 = 1	1	0	0	0
9 (0×9)	H9 = 1	1	0	0	1
10 (0xA)	H10 = 1	1	0	1	0
11 (0xB)	H11 = 1	1	0	1	1
12 (0xC)	H12 = 1	1	1	0	0
13 (0xD)	H13 = 1	1	1	0	1
14 (0xE)	H14 = 1	1	1	1	0
15 (0xF)	H15 = 1	1	1	1	1

23. NAND Gate using NOR Gate (Universal Gate Conversions · Lab Page 14)

Function: Constructs a NAND gate using 4 NOR gates: two NOR inverters invert A and B, a third NOR gate forms A·B, and a fourth NOR inverter complements the output to ~(A·B).

Boolean Expression: $Y = \overline{A \cdot B} = \overline{A} + \overline{B}$ (via 4 NOR gates)

A	B	A . B	Y = ~(A . B)
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

24. NOR Gate using NAND Gate (Universal Gate Conversions · Lab Page 14)

Function: Constructs a NOR gate using 4 NAND gates: two NAND inverters produce ~A and ~B, a third NAND gate forms A+B, and a fourth NAND inverter complements the output to ~(A+B).

Boolean Expression: $Y = \overline{A + B} = \overline{A} \cdot \overline{B}$ (via 4 NAND gates)

A	B	A + B	Y = ~(A + B)
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

25. OR Gate using NAND Gate (Universal Gate Conversions · Lab Page 15)

Function: Realizes the logical OR operation using 3 NAND gates by applying De Morgan's theorem: two NAND inverters invert inputs A and B, feeding a third 2-input NAND gate.

Boolean Expression: $Y = \overline{A \cdot B} = (A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B) = A + B$

A	B	~A	~B	Y = A + B
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	1

26. OR Gate using NOR Gate (Universal Gate Conversions · Lab Page 15)

Function: Realizes the logical OR operation using 2 NOR gates by following a 2-input NOR gate with a single-input NOR inverter.

Boolean Expression: $Y = \overline{A + B} = (A \text{ NOR } B) \text{ NOR } (A \text{ NOR } B) = A + B$

A	B	~(A + B)	Y = A + B
0	0	1	0
0	1	0	1
1	0	0	1
1	1	0	1

27. SR Flip-Flop using NAND Gate (Latches & Flip-Flops · Lab Page 15)

Function: Clocked Set-Reset flip-flop built from 4 NAND gates (two input steering NAND gates controlled by CLK and two cross-coupled latch NAND gates).

Boolean Expression: $Q(t+1) = S + \overline{R} \cdot Q(t)$ (with $S \cdot R = 0$ when CLK = 1)

CLK	S	R	Q(t+1)	Q_bar(t+1)	Operating Mode
0	X	X	Q(t)	Q_bar(t)	Hold (Clock Low)
1	0	0	Q(t)	Q_bar(t)	Hold (No Change)
1	0	1	0	1	Reset (Q = 0)
1	1	0	1	0	Set (Q = 1)
1	1	1	1*	1*	Invalid / Forbidden (S=R=1)

28. SR Flip-Flop using NOR Gate (Latches & Flip-Flops · Lab Page 16)

Function: Clocked Set-Reset flip-flop realized exclusively with NOR gates: NOR steering stages gate S and R with CLK to drive a cross-coupled active-high NOR latch.

Boolean Expression: $Q(t+1) = S + \overline{R} \cdot Q(t)$ (with $S \cdot R = 0$ when CLK = 1)

CLK	S	R	Q(t+1)	Q_bar(t+1)	Operating Mode
0	X	X	Q(t)	Q_bar(t)	Hold (Clock Low)
1	0	0	Q(t)	Q_bar(t)	Hold (No Change)
1	0	1	0	1	Reset (Q = 0)
1	1	0	1	0	Set (Q = 1)
1	1	1	0*	0*	Invalid / Forbidden (S=R=1)

29. SR Latch using NAND Gate (Latches & Flip-Flops · Lab Page 16)

Function: Asynchronous active-low bistable latch formed by two cross-coupled 2-input NAND gates driven by S_bar and R_bar.

Boolean Expression: $Q = \overline{S_bar \cdot Q_bar}$, $Q_bar = \overline{R_bar \cdot Q}$

S_bar	R_bar	Q(t+1)	Q_bar(t+1)	Operating Mode
0	0	1*	1*	Invalid / Forbidden (Active-Low Both 0)
0	1	1	0	Set (Q = 1)
1	0	0	1	Reset (Q = 0)
1	1	Q(t)	Q_bar(t)	Hold (Memory State)

30. SR Latch using NOR Gate (Latches & Flip-Flops · Lab Page 16)

Function: Asynchronous active-high bistable latch formed by two cross-coupled 2-input NOR gates driven by R (top gate) and S (bottom gate).

Boolean Expression: $Q = \overline{R + Q_bar}, \quad Q_bar = \overline{S + Q}$

S	R	Q(t+1)	Q_bar(t+1)	Operating Mode
0	0	Q(t)	Q_bar(t)	Hold (Memory State)
0	1	0	1	Reset (Q = 0)
1	0	1	0	Set (Q = 1)
1	1	0*	0*	Invalid / Forbidden (Both 1)

31. Seven Segment Display (3-Bit) (Encoders & Decoders · Lab Page 17)

Function: Decodes a 3-bit octal input (A2, A1, A0 representing digits 0 through 7) into the 7 active-high display segment signals (a, b, c, d, e, f, g).

Boolean Expression: $a = A_1 + A_2A_0 + \overline{A_2}\,\overline{A_0}, \quad b = \overline{A_2} + \overline{A_1}\,\overline{A_0} + A_1A_0, \quad c = A_2 + \overline{A_1} + A_0, \quad e = \overline{A_2}\,\overline{A_0} + A_1\overline{A_0}$

Digit	A2	A1	A0	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
1	0	0	1	0	1	1	0	0	0	0
2	0	1	0	1	1	0	1	1	0	1
3	0	1	1	1	1	1	1	0	0	1
4	1	0	0	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1	0	1	1
6	1	1	0	1	0	1	1	1	1	1
7	1	1	1	1	1	1	0	0	0	0

32. XNOR Gate using NAND Gate (Universal Gate Conversions · Lab Page 18)

Function: Implements the Equivalence (XNOR) function using 5 NAND gates: a 4-NAND XOR stage followed by a 5th NAND inverter.

Boolean Expression: $Y = \overline{A \oplus B} = (A \cdot B) + (\overline{A} \cdot \overline{B})$

A	B	A (+) B	Y = ~(A (+) B)
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

33. XNOR Gate using NOR Gate (Universal Gate Conversions · Lab Page 18)

Function: Implements the Equivalence (XNOR) function using the minimal 4-NOR topology: n1 = ~(A+B), n2 = ~(A+n1), n3 = ~(B+n1), and Y = ~(n2+n3).

Boolean Expression: $Y = \overline{A \oplus B} = (A \cdot B) + (\overline{A} \cdot \overline{B})$ (via 4 NOR gates)

A	B	n1=~(A+B)	Y = ~(A (+) B)
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	1

34. XOR Gate using NAND Gate (Universal Gate Conversions · Lab Page 18)

Function: Implements the Exclusive-OR (XOR) function using the classic 4-NAND architecture: n1 = ~(A·B), n2 = ~(A·n1), n3 = ~(B·n1), and Y = ~(n2·n3).

Boolean Expression: $Y = A \oplus B = (A \cdot \overline{B}) + (\overline{A} \cdot B)$ (via 4 NAND gates)

A	B	n1=~(A.B)	Y = A (+) B
0	0	1	0
0	1	1	1
1	0	1	1
1	1	0	0

35. XOR Gate using NOR Gate (Universal Gate Conversions · Lab Page 18)

Function: Implements the Exclusive-OR (XOR) function using 5 NOR gates by cascading the 4-NOR XNOR stage into a 5th NOR inverter.

Boolean Expression: $Y = A \oplus B = (A \cdot \overline{B}) + (\overline{A} \cdot B)$ (via 5 NOR gates)

A	B	~(A (+) B)	Y = A (+) B
0	0	1	0
0	1	0	1
1	0	0	1
1	1	1	0

36. AND Gate (Basic Logic Gates · Lab Page 19)

Function: Basic 2-input logical conjunction gate. Output Y is HIGH (1) if and only if both inputs A and B are HIGH (1).

Boolean Expression: $Y = A \cdot B$

A	B	Y = A . B
0	0	0
0	1	0
1	0	0
1	1	1

37. OR Gate (Basic Logic Gates · Lab Page 19)

Function: Basic 2-input logical disjunction gate. Output Y is HIGH (1) when at least one input (A or B) is HIGH (1).

Boolean Expression: $Y = A + B$

A	B	Y = A + B
0	0	0
0	1	1
1	0	1
1	1	1

38. NOT Gate (Basic Logic Gates · Lab Page 19)

Function: Single-input logical inverter. Produces the complement (~A) of its input signal A.

> **Boolean Expression: $Y = \overline{A}$**

A	Y = ~A
0	1
1	0

39. XOR Gate (Basic Logic Gates · Lab Page 19)

Function: 2-input Exclusive-OR gate. Output Y is HIGH (1) when inputs A and B have different logic levels (odd number of 1s).

> **Boolean Expression: $Y = A \oplus B = A\cdot\overline{B} + \overline{A}\cdot B$**

A	B	Y = A (+) B
0	0	0
0	1	1
1	0	1
1	1	0

40. XNOR Gate (Basic Logic Gates · Lab Page 19)

Function: 2-input Exclusive-NOR (Equivalence) gate. Output Y is HIGH (1) when both inputs A and B have the same logic level.

> **Boolean Expression: $Y = \overline{A \oplus B} = A\cdot B + \overline{A}\cdot\overline{B}$**

A	B	Y = ~(A (+) B)
0	0	1
0	1	0
1	0	0
1	1	1

41. NOR Gate (Basic Logic Gates · Lab Page 20)

Function: Universal 2-input NOT-OR gate. Output Y is HIGH (1) only when both inputs A and B are LOW (0).

> **Boolean Expression: $Y = \overline{A + B}$**

A	B	Y = ~(A + B)
0	0	1
0	1	0
1	0	0
1	1	0

42. NAND Gate (Basic Logic Gates · Lab Page 20)

Function: Universal 2-input NOT-AND gate. Output Y is LOW (0) only when both inputs A and B are HIGH (1).

> **Boolean Expression: $Y = \overline{A \cdot B}$**

A	B	Y = ~(A . B)
0	0	1
0	1	1
1	0	1
1	1	0

43. T Flip-Flop (Latches & Flip-Flops · Lab Page 20)

Function: Single-input Toggle flip-flop derived from the JK flip-flop by tying J = K = T. When CLK = 1 and T = 0, the state holds; when CLK = 1 and T = 1, the output toggles.

Boolean Expression: $Q(t+1) = T \oplus Q(t) = T\cdot\overline{Q(t)} + \overline{T}\cdot Q(t)$ (when CLK = 1)

CLK	T	Q(t)	Q(t+1)	Q_bar(t+1)	Operating Mode
0	X	0	0	1	Hold (Clock Low)
0	X	1	1	0	Hold (Clock Low)
1	0	0	0	1	Hold (No Toggle, T=0)
1	0	1	1	0	Hold (No Toggle, T=0)
1	1	0	1	0	Toggle (0 → 1)
1	1	1	0	1	Toggle (1 → 0)

44. Implementation of Three-Variable Boolean Expression using NOR Gates

(Boolean Expressions · Lab Page 20)

Function: Implements a 3-variable Product-of-Sums (POS) Boolean expression exclusively with universal NOR gates (NOR inverters and a 2-level NOR-NOR array).

Boolean Expression: $Y(A, B, C) = (A + B) \cdot (\overline{A} + C) \cdot (B + \overline{C})$

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

45. Implementation of Four-Variable Boolean Expression using Basic Gates

(Boolean Expressions · Lab Page 21)

Function: Implements a 4-variable Sum-of-Products (SOP) expression using 2 NOT gates, 3 two-input AND gates, and 1 three-input OR gate.

> **Boolean Expression: $Y(A, B, C, D) = (A \cdot B) + (C \cdot D) + (\overline{A} \cdot \overline{C})$**

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

46. Implementation of Four-Variable Boolean Expression using NAND Gate

(Boolean Expressions · Lab Page 22)

Function: Implements the 4-variable SOP Boolean expression strictly using universal NAND gates (2 NAND inverters, 3 first-level NAND gates, and 1 output NAND gate).

Boolean Expression: $Y = \overline{A \cdot B \cdot C \cdot D \cdot A \cdot C} = (A \cdot B) + (C \cdot D) + (\overline{A} \cdot \overline{C})$

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

47. NOT Gate using NAND Gate (Universal Gate Conversions · Lab Page 22)

Function: Configures a 2-input NAND gate as an inverter by tying both input terminals together to input signal A.

Boolean Expression: $Y = \overline{A \cdot A} = \overline{A}$

A	Y = ~A (A NAND A)
0	1
1	0

48. NOT Gate using NOR Gate (Universal Gate Conversions · Lab Page 22)

Function: Configures a 2-input NOR gate as an inverter by tying both input terminals together to input signal A.

> **Boolean Expression: $Y = \overline{A + A} = \overline{A}$**

A	Y = ~A (A NOR A)
0	1
1	0

49. Implementation of Two-Variable Boolean Expression using Basic Gates

(Boolean Expressions · Lab Page 23)

Function: Realizes the 2-variable SOP expression Y = A·~B + ~A·B using basic gates (2 NOT gates, 2 AND gates, and 1 OR gate).

> **Boolean Expression: $Y(A, B) = (A \cdot \overline{B}) + (\overline{A} \cdot B)$**

A	B	~A	~B	Y
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	0

50. Implementation of Two-Variable Boolean Expression using NAND Gate

(Boolean Expressions · Lab Page 23)

Function: Realizes the 2-variable SOP expression Y = A·~B + ~A·B exclusively using universal NAND gates (2 NAND inverters and 3 two-input NAND gates).

> **Boolean Expression: $Y = \overline{A \cdot B \cdot A \cdot B} = (A \cdot \overline{B}) + (\overline{A} \cdot B)$**

A	B	~A	~B	Y
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	0